Exposición final — Tu RAG, con cara

Consigna de las exposiciones del viernes 6/11 y el viernes 13/11

En las dos últimas clases cada uno muestra el RAG que construyó: el de la clase 8, sobre su propio corpus, con los vectores en Postgres. Para la exposición le suma una cosa más: una cara. Hasta ahora el sistema se usaba desde la terminal; para el 6 o el 13 tiene que tener un front, la forma en que una persona cualquiera le haría una pregunta. Cuál, lo elegís vos.

1. El front: como quieras, pero que sea tuyo

No hay un formato obligatorio. Se puntúa la creatividad: que la forma de preguntar tenga que ver con tu corpus y con quién lo usaría. Algunas ideas para arrancar, no para copiar:

- Una página web con la identidad de tu tema: si tu corpus es el reglamento de un club, que parezca la cartelera del club.
- Un bot de Telegram, Discord o WhatsApp al que le escribís como a una persona.
- Un personaje que responde: el bibliotecario, el árbitro, el técnico de soporte, con su tono.
- Algo que no sea un chat: un buscador con fichas, un juego de preguntas sobre el corpus, una extensión del navegador, una app de escritorio.
- Una terminal, si querés, pero con onda: colores, menú, historial.

Las tres reglas

1. **Tiene que usar tu RAG de verdad.** La misma búsqueda en Postgres, el mismo umbral y la respuesta con fuente. Un chat que le pregunta al modelo directo, sin tus fragmentos, no cuenta.
2. **La clave de API no va en el navegador.** Lo vimos en la clase 6: cualquiera la lee con F12. Si tu front es una página web, necesita un servidor en Node que haga la consulta. En el Anexo hay uno mínimo, listo para copiar.
3. **Tiene que mostrar de dónde sale la respuesta.** La fuente (archivo y sección) visible para el que pregunta, como quieras mostrarla.

> Podés usar IA para hacer el front: pedíselo a un agente, a un chat, a quien quieras. Para el front se permite cualquier librería o framework (React, Vue, lo que sea). El núcleo del RAG sigue siendo el tuyo, el de la clase 8: nada de LangChain ni bases vectoriales dedicadas. Eso sí: lo que muestres lo tenés que poder explicar.

2. Qué tenés que mostrar

No hay un tiempo fijo: el que necesites para mostrar todo esto funcionando, en vivo.

#	Qué	Cómo
1	Tu corpus	De qué es, por qué lo elegiste y quién lo usaría.
2	El front	Cómo se usa y por qué lo pensaste así.

#	Qué	Cómo
3	Una pregunta que funciona	En vivo, con su fuente. Abrí el documento y mostrá que la fuente dice lo que dijo el modelo.
4	Una pregunta que no está	El sistema tiene que decir que no, sin inventar.
5	Una pregunta que falla	Todo sistema falla en algo. Mostrá una y explicá si falló la búsqueda o el modelo, y por qué.
6	Tus estrategias	Cómo troceaste (tu marca), qué umbral elegiste y con qué recta, qué le cambiaste al prompt, y cualquier otra decisión que hayas tomado. Contá qué probaste y descartaste.

> Tu Ficha 4 (la recta del umbral) y tu Ficha 5 (las cuatro preguntas) de la clase 8 son la base de los puntos 3 a 6.

3. Preguntas que te puedo hacer

Después de la demo te voy a hacer algunas preguntas, unas técnicas y otras sobre cómo funciona un RAG. Pueden ser de esta lista o parecidas:

Sobre tu sistema

- ¿Qué parte corre en tu compu y qué parte en la nube? ¿Por qué ese reparto?
- ¿Qué pasa si cambiás el modelo de embeddings y no volvés a indexar?
- ¿Qué pasa si corrés indexar.mjs dos veces? ¿Cómo lo resolviste?
- ¿Por qué la clave de API no puede ir en el navegador? ¿Dónde está en tu front?
- ¿Por qué el ORDER BY va por la distancia y no por la similitud?
- ¿Qué error preferiste cometer con tu umbral, y por qué?
- Mostrame en el código dónde se arma el prompt.

Sobre cómo funciona un RAG

- ¿Qué es un embedding? ¿Por qué textos que no comparten ninguna palabra pueden quedar cerca?
- ¿Qué mide la similitud coseno?
- ¿Por qué no alcanza con pegarle el corpus entero al modelo en cada pregunta?
- ¿Cómo encuentra HNSW los fragmentos más cercanos sin mirarlos a todos? ¿Por qué es aproximado?
- ¿Quién evita que el modelo invente: la búsqueda, el umbral o el prompt?
- ¿Qué le falta a tu sistema para usarse en serio? (Pista: hay cuatro cosas que nombramos al cierre de la clase 8.)

4. Cómo se evalúa

Criterio	Qué se mira	Puntos
El RAG funciona	Responde en vivo sobre tu corpus, con fuente verificable, y dice que no cuando la respuesta no está.	25
Entendés cómo funciona	Respondés las preguntas técnicas y conceptuales con tus palabras, sin leer.	25
Estrategias explicadas	Sabés por qué troceaste así, por qué ese umbral y qué cambiaste del prompt, con evidencia (tu recta, tus pruebas).	20
Creatividad del front	Que sea original, que tenga que ver con tu corpus y con quién lo usaría, y que use el RAG de verdad.	20
Honestidad con las fallas	Mostrás una pregunta que falla y la diagnosticás: búsqueda o modelo, y por qué.	10
Total		**100**

Extras que suman

No son obligatorios. Cualquiera de estos, bien explicado, puede subir la nota dentro de los criterios de estrategias o de front:

- Que el front muestre los fragmentos recuperados y su similitud, no solo la respuesta.
- Historial de la conversación.
- Más de un documento en el corpus, y una pregunta que necesite datos de los dos.
- Más preguntas de prueba que las seis, y cuántas acierta: una medición propia.
- Haber comparado dos troceados o dos umbrales, y poder decir cuál anduvo mejor y por qué.

5. Cuándo y cómo

Dato	Detalle
Fechas	Viernes 6/11 y viernes 13/11, en el horario de clase.
Quién expone cada día	Anotate en la planilla que comparte el docente.
Individual	Cada uno presenta su propio RAG.
Se presenta desde	Tu compu, con tu sistema corriendo en vivo.

Antes de que te toque

1. Docker Desktop abierto y la base prendida: docker start rag-db.
2. Ollama abierto.
3. El .env con la clave de Google, y conexión a internet para la respuesta.
4. El front corriendo y probado esa misma mañana.

5. Tus preguntas anotadas: la que funciona, la que no está y la que falla.
6. El documento abierto en la sección que vas a mostrar como fuente.

> **ATENCIÓN**
>
> Plan B: si el día de la exposición falla la red o la compu, tené capturas de pantalla o un video corto del sistema funcionando. No reemplaza la demo, pero salva la exposición. Y probá todo el día anterior: «en mi casa andaba» no suma puntos.

Anexo — Un front web mínimo, para arrancar

Es opcional: es el esqueleto más chico posible, para que veas cómo se conecta una página con tu RAG sin que la clave salga de tu compu. Tomalo como punto de partida y cambialo todo lo que quieras. Los tres archivos van en tu carpeta rag-propio.

rag.mjs — tu RAG convertido en una función

Es lo mismo que hace preguntar.mjs, pero en lugar de imprimir, devuelve la respuesta y las fuentes.

CÓDIGO — rag.mjs

```
// El RAG de la clase 8 convertido en una función, para usarlo desde cualquier front.
import { readFile } from "node:fs/promises";
import { embedding, responder } from "./api.mjs";
import { buscar } from "./buscar.mjs";

const PLANTILLA = await readFile("prompt.txt", "utf8");
const UMBRAL = Number(process.env.UMBRAL ?? 0);

export async function preguntarRAG(pregunta) {
  const resultados = await buscar(await embedding(pregunta), 3);
  const utiles = resultados.filter(r => r.similitud >= UMBRAL);
  const fuentes = utiles.map(r => ({ fuente: r.fuente, seccion: r.seccion, similitud:
r.similitud }));
  if (utiles.length === 0) return { respuesta: "No lo encuentro en los documentos.",
fuentes };

  const etiqueta = r => `${r.fuente} · ${r.seccion ?? `fragmento ${r.posicion}`}`;
  const fragmentos = utiles.map(r => `--- ${etiqueta(r)} ---\n${r.texto}`).join("\n\
n");
  const prompt = PLANTILLA.replace("{{fragmentos}}",
fragmentos).replace("{{pregunta}}", pregunta);
  return { respuesta: await responder(prompt), fuentes };
}
```

servidor.mjs — recibe la pregunta y devuelve la respuesta

CÓDIGO — servidor.mjs

```
// Un servidor mínimo: tu front le manda la pregunta y recibe la respuesta.
// La clave de Google queda acá, en tu compu, nunca en el navegador.
//   node --env-file=.env servidor.mjs      ->  http://localhost:3000
import http from "node:http";
import { readFile } from "node:fs/promises";
```

```
import { preguntarRAG } from "./rag.mjs";

http.createServer(async (req, res) => {
  try {
    if (req.method === "POST" && req.url === "/preguntar") {
      let cuerpo = "";
      for await (const pedazo of req) cuerpo += pedazo;
      const { pregunta } = JSON.parse(cuerpo);
      res.writeHead(200, { "Content-Type": "application/json; charset=utf-8" });
      res.end(JSON.stringify(await preguntarRAG(pregunta)));
    } else {
      // cualquier otra dirección: tu página (index.html, en la misma carpeta)
      res.writeHead(200, { "Content-Type": "text/html; charset=utf-8" });
      res.end(await readFile("index.html"));
    }
  } catch (e) {
    res.writeHead(500, { "Content-Type": "application/json; charset=utf-8" });
    res.end(JSON.stringify({ error: e.message }));
  }
}).listen(3000, () => console.log("Listo: http://localhost:3000"));
```

index.html — la página más fea del mundo, para que la mejores

CÓDIGO — index.html

```
<!doctype html>
<meta charset="utf-8">
<title>Mi RAG</title>
<input id="pregunta" placeholder="Preguntá algo" size="50">
<button id="enviar">Preguntar</button>
<pre id="salida"></pre>
<script>
  document.getElementById("enviar").onclick = async () => {
    const pregunta = document.getElementById("pregunta").value;
    const res = await fetch("/preguntar", { method: "POST", body:
JSON.stringify({ pregunta }) });
    const { respuesta, fuentes } = await res.json();
    document.getElementById("salida").textContent =
      respuesta + "\n\n" + fuentes.map(f => `${f.fuente} · ${f.seccion}  ($
{f.similitud.toFixed(3)})`).join("\n");
  };
</script>
```

Para probarlo

CÓDIGO — terminal

```
node --env-file=.env servidor.mjs
```

Abrí http://localhost:3000 en el navegador. Si tu front es un bot o una app, la idea es la misma: el bot llama a preguntarRAG(pregunta) de rag.mjs y muestra lo que devuelve.